SUPERVISED LEARNING PROJECT REPORT

(Project Term October - December 2025)

(Fraud Detection System for Credit Card Transactions)

Submitted by

Name: T Aryan Chand **Registration No: 12310528**
Name: Akash K A **Registration No: 12311479**
Name: S K Bashu **Registration No: 12304465**
Name: Yashwanth **Registration No: 12308264**

Project Group Number: 30

Course Code: INT 395

Under the Guidance of

Dr. Geetika Sethi

School of Computer Science and Engineering

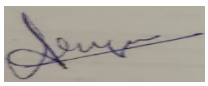
LOVELY PROFESSIONAL UNIVERSITY

DECLARATION

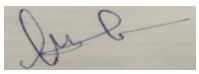
We hereby declare that the project work entitled “Fraud Detection System for Credit Card Transactions” is an authentic record of our own work carried out as requirements of Capstone Project for the award of B. Tech degree in Computer Science and Engineering (AI/ML) from Lovely Professional University, Phagwara, under the guidance of (Geetika Sethi), during October to November 2025. All the information furnished in this capstone project report is based on our own intensive work and is genuine.

Project Group Number: 30

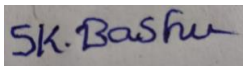
Name of Student: T Aryan Chand
Registration Number: 12310528



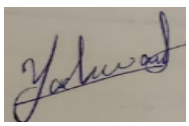
Name of Student: Akash K A
Registration Number: 12311479



Name of Student: S K Bashu
Registration Number: 12304465



Name of Student: Yashwanth
Registration Number: 12308264



ACKNOWLEDGEMENT

We would like to express our profound gratitude for the opportunity to work on this project titled **"Credit Card Fraud Detection System Using Machine Learning."** This project has provided us with invaluable exposure to real-world financial security challenges, as well as a deeper understanding of applied machine learning techniques, imbalanced classification, and predictive analytics.

First and foremost, we extend our heartfelt thanks to our mentor and project guide, **Dr. Geetika Sethi**, for her constant encouragement, insightful feedback, and expert guidance throughout the duration of this project. Her support helped us refine our methodology, fine-tune our models, and understand the practical implications of fraud detection systems in modern digital banking environments. Her patience, constructive criticism, and academic mentorship were pivotal to our project's success.

We are grateful to the **School of Computer Science and Engineering, Lovely Professional University**, for providing us with the academic infrastructure, computational resources, and a highly conducive learning environment that enabled us to explore, experiment, and innovate.

We also acknowledge the availability of high-quality open-source datasets and tools, particularly the **Kaggle Credit Card Fraud Dataset**, which served as the foundation for our research. Additionally, we thank the developers and contributors of **Python, Scikit-Learn, NumPy, Pandas, Matplotlib, Seaborn**, and other open-source libraries for empowering us with robust tools to implement and analyse machine learning models effectively.

We extend our appreciation to our classmates, peers, and project group members for their teamwork, discussions, and collaborative spirit. Their support helped us overcome various technical hurdles and refine our approach across different stages of the project.

Finally, we would like to express our deepest gratitude to our parents and families for their unwavering support, motivation, and encouragement. Their belief in our abilities has been a constant source of strength.

With sincere appreciation,
we dedicate the successful completion of this project to everyone who contributed to its development in any capacity.

TABLE OF CONTENTS

1. Introduction

2.Profile of the Problem

3.Prior Art and Core Technical Gap

4.Problem Analysis

5.Design

6.Testing

7. Implementation

8.User Manual

9.Limitations

10. Source Code and Snapshots

1. Introduction

The rapid expression of digital payments systems has significantly increased the convenience of financial transactions. However, this expansion has also resulted in a dramatic rise in **credit card fraud,** causing financial losses for banks, merchants and customers.

Traditional rule-based fraud detection systems are no longer sufficient due to:

- Evolving fraud patterns
- Increasing transaction volumes
- Sophisticated attack behaviour

Machine Learning (ML) has emerged as a powerful solution capable of learning hidden patterns in transaction data and detecting anomalies with high accuracy. This project aims to develop a **supervised ML based fraud detection system** using the widely used European Credit Card Dataset (Kaggle) containing:

- PCA-transformed numerical features (V1-V28)
- Time and Amount
- Class = target label (0 = genuine, 1 = fraud)

The objective of this project is to design, implement, evaluate and compare multiple ML algorithms to identify fraudulent transactions with high recall and F1-score ensuring minimal false negatives.

2. Profile of the Problem

2.1 Problem Definition

The central problem addressed in this project is the binary classification of credit card transaction into:

- Fraudulent (Class = 1)
- Legitimate (Class = 0)

The system must learn patterns from historical data and accurately detect frauds even when:

- Fraud instances are extremely rare
- Fraud patterns are continuously evolving
- Transaction behaviour varies widely across users

Challenges Identified

A. Extreme Class Imbalance

- Only 0.17% of the dataset consists of fraud cases.
- Most ML models naturally bias toward predicting the majority class (non-fraud).
- High accuracy becomes misleading since predicting all transactions as “non-fraud” gives 00.8% accuracy but 0% recall.

B. Hidden Fraud Patterns

The dataset uses **PCA-transformed** features to anonymize transaction information.

- Features V1-V28 are not interpretable.
- Manual feature engineering becomes difficult.
- The model must uncover abstract relationships autonomously.

C. Real-Time Prediction Requirement

Fraud must be detected instantly, before or during the transaction.

This demands:

- High precision
- Very high recall
- Low-latency inference

D. Cost of False Negatives are Extremely High

A single missed fraud can result in:

- Financial loss
- Chargeback penalties
- Damage to customer trust

Thus, recall is treated as the primary metric in this project.

2.2 Scope of the Study

To ensure methodological rigor and reproducibility, the scope of the project is defined as follows:

A. Dataset and Data Integrity

- Only the official Kaggle Credit Card Fraud Dataset is used.
- No external or synthetic financial datasets are introduced unless used for balancing.
- A strict 80-20 stratified split ensures fair evaluation.

B. Comparative Model Evaluation

We implement and compare multiple ML models:

- Logistic Regression
- Naïve Bayes
- K-Nearest Neighbors
- Support Vector Machine
- Decision Tree
- Random Forest

The goal is to determine which model provides the highest recall while maintaining string precision and F1-score.

C. Success Criteria

A model is deemed successful if it achieves:

- Recall > 90%
- F1-score > 90%
- Robustness against class imbalance
- Consistent results across test sets

D. Output Deliverables

The study produces:

- A trained fraud detection model
- Evaluation metrics (Accuracy, Recall, Precision, F1, ROC-AUC)
- Visualizations (Confusion matrix, ROC curve, EDA charts)
- A prediction function for new transactions
- Saved model artifacts (.pkl)

3. Prior Art and Core Technical Gap

3.1 Existing fraud Detection Approaches

Credit card fraud detection has traditionally relied on rule-based expert systems, where banks define a set of fixed rules to flag suspicious activity. These rules often monitor patterns such as unusually high transaction amounts, multiple rapid purchases within a short period, transactions originating from geographically distant locations, or sudden deviations from a customer's typical spending habits. Although these rule engines are simple to implement and easy to interpret, they suffer from severe limitations. Fraudsters frequently evolve their strategies, rendering static rules ineffective. Furthermore, rigid rule-based systems tend to produce a high number of false positives, unnecessarily blocking legitimate customer transactions, and increasing customer dissatisfaction. They also fail to detect sophisticated fraud patterns that do not violate predefined conditions.

As fraud tactics became more complex, the industry shifted towards modern data-driven approaches,

particularly **Machine Learning (ML)**. ML models such as Logistic Regression, Decision Trees, Random Forests, Support Vector Machines, and Gradient Boosting algorithms like XGBoost or LightGBM are capable of learning nuanced correlations in transactional data. These models capture subtle fraud indicators that might not be apparent through manual rule creation. Unlike traditional systems, ML-based detectors continuously improve as more data becomes available, making them far more adaptive to changing fraud patterns.

Beyond classical machine learning, the field has witnessed rapid growth in Deep Learning approaches. Models like Autoencoders are widely used for anomaly detection, especially when fraud labels are scarce. Recurrent neural networks such as LSTMs can learn temporal purchasing behaviours, analysing sequential spending patterns to detect irregularities. Convolutional Neural Networks (CNNs) have also been applied to structured transaction data by treating feature interactions as “spatial patterns.” Despite their promising performance, deep learning models typically require large datasets, extensive computational power, and careful tuning, which may not always be feasible for real-time fraud detection systems.

To further enhance accuracy, many financial institutions now rely on hybrid and ensemble-based approaches. These systems combine multiple ML models or integrate traditional rules with advanced algorithms to reduce false alarms and improve fraud catch rate. Techniques such as bagging, boosting, and stacking using algorithms like Random Forest, XGBoost, or AdaBoost, have demonstrated exceptional performance, especially in handling imbalanced data. Ensemble methods reduce variance, improve model robustness, and generate more reliable predictions across diverse fraud scenarios.

3.2 Limitations of Prior Solutions

Although the landscape of fraud detection has significantly evolved, several important limitations still exist. One of the most persistent challenges is the extreme class imbalance inherent to fraud datasets. With fraudulent transactions representing less than 0.2% of total transactions, most ML models tend to be biased toward predicting the majority class (non-fraud). As a result, even models with very high overall accuracy may fail to detect actual fraud, making them unsuitable for practical deployment.

Another major limitation involves interpretability. In many publicly available datasets, including the widely used Kaggle dataset employed in this project, the transaction features have been transformed using PCA for privacy protection. While PCA ensures anonymity, it produces high-dimensional, non-interpretable numerical features (V1–V28), making it challenging to understand the reasoning behind model predictions. This lack of explainability is a serious concern in banking environments where compliance and transparency are mandatory.

Additionally, fraud patterns are dynamic and evolve continuously. Fraudsters adapt to new security measures, develop sophisticated evasion techniques, and mimic legitimate user behaviour to bypass detection systems. Static or infrequently retrained models degrade quickly over time, leading to higher false negatives. Maintaining a high-performance fraud detection model therefore requires periodic retraining, threshold tuning, and continuous monitoring.

The problem of overfitting also poses a risk, especially when dealing with a small number of fraud samples. Some models memorize specific fraud examples rather than learning generalizable fraud behaviour. This results in good performance on training data but poor detection on unseen transactions.

Finally, the cost of false negatives is extremely high in fraud detection. A missed fraudulent transaction leads to immediate financial loss, potential reimbursement costs, chargebacks, and long-term damage to customer trust. Therefore, the primary objective of a fraud detection system must be to maximize recall, even if it requires tolerating a reasonable number of false positives.

3.3 Core Technical Gap Identified

After surveying existing research and industry solutions, several core technical gaps become evident. The first and most critical gap is the lack of high-recall models. Many studies and implementations focus on achieving high accuracy, which is misleading in heavily imbalanced datasets. In real-world fraud detection, recall the ability to correctly identify fraud is far more important than accuracy.

The second major gap lies in inefficient data preprocessing pipelines. Many prior studies do not incorporate essential transformations such as scaling transaction amounts, deriving time-based features, or engineering behaviour driven attributes.

In the Kaggle dataset, for example, extracting features like transaction hour or scaling the Amount field significantly affects model quality, yet many existing works overlook these steps.

Another gap is the absence of standardized benchmarking across multiple ML algorithms. Many research papers evaluate only one or two models in isolation, offering limited insight into comparative performance under identical conditions. This lack of benchmarking restricts the ability to select the most suitable model for deployment.

A fourth critical gap pertains to the lack of end-to-end systems that include every stage of a production-ready fraud detection pipeline. Most prior works emphasize only model training but fail to present complete workflows involving exploratory data analysis, feature engineering, pipeline creation, evaluation metrics, model comparison frameworks, saving trained models, visualization of results, or integration-ready outputs.

This project directly addresses these gaps by providing a holistic, fully structured, end-to-end machine learning-based fraud detection system, prioritizing recall-based evaluation and including comprehensive preprocessing, visualization, benchmarking, and deployment readiness.

4. Problem Analysis

4.1 Understanding the Dataset

The dataset contains:

Feature Type	Description
Time	Seconds elapsed between current and first transaction
Amount	Transaction amount
V1-V28	PCA-transformed numerical components
Class	Target (0 = Genuine, 1 = Fraud)

Dataset Size:

- 2,84,807 transactions
- 492 fraud transactions
- Imbalance: 0.17%

4.2 Challenges in Data Distribution

A. Highly Skewed Target Variable

Class	Count
0 (Genuine)	2,84,315
1 (Fraud)	492

B. Non-interpretable PCA Features

Features do not have business context.
Thus, modelling relies solely on:

- Mathematical relationships
- Statistical patterns

C. variable Transaction Behaviours

Different users have different transaction habits. Fraudulent mimic normal user behaviour and is harder to detect.

5. Design

The design phase of the Credit Card Fraud Detection System forms the structural and conceptual foundation upon which the entire project is built. Since fraud detection requires high accuracy, reliability, and real-time performance, the system design emphasizes modularity, data efficiency, and robust machine learning integration. The architecture is constructed to ensure smooth data flow, scalable preprocessing, clear separation of concerns, and support for future deployment in web or cloud-based environments.

The overall system is designed as a pipeline consisting of several interconnected modules data acquisition, preprocessing, exploratory analysis, feature engineering, model training, evaluation, and final prediction. Each module is built independently but integrated cohesively to form an end-to-end detection system. This modular structure ensures that updates or improvements in one component such as modifying the preprocessing logic or replacing the ML model can be incorporated without affecting the rest of the workflow.

At a high level, the system follows a typical machine learning architecture where raw data is ingested, cleaned, transformed, and passed into classification algorithms that learn patterns distinguishing legitimate transactions from fraudulent ones. Because credit card fraud patterns evolve constantly, the design incorporates flexible components such as pipeline-based preprocessing, scalable feature extraction, and reusable evaluation functions. These components make the system highly adaptable to new datasets or changes in fraud behaviour.

From a structural perspective, the design also ensures that the model is not tightly coupled to the dataset. To achieve this, the system uses configurable parameters and dynamic feature-handling mechanisms, enabling smooth transition from experimental environments such as Jupyter/Colab notebooks to production ready systems. The use of the Scikit-learn Pipeline architecture further strengthens this design by encapsulating multiple preprocessing and modelling steps into a single object, preventing data leakage and ensuring consistent transformations during training and testing.

5.1 System Architecture

The architecture of the fraud detection system consists of sequential stages, each responsible for a critical part of the detection process. The design reflects a linear yet flexible data pipeline.

The process begins with Data Acquisition, where the Kaggle dataset is imported through KaggleHub or local storage. Once the data is ingested, it moves into the Data Preprocessing Module, which cleans the dataset, handles missing values, scales numerical attributes, and engineers new features such as the transaction hour. This enhances the dataset's quality and ensures that all models receive standardized numerical input.

The next stage is Exploratory Data Analysis (EDA). Although not directly involved in prediction, EDA is vital to understanding data distribution, class imbalance, and hidden correlations. The architectural design incorporates plots such as correlation heatmaps, fraud distributions, amount distributions, and hourly transaction patterns. Insights derived during this stage influence later decisions such as algorithm selection and feature engineering strategy.

Following EDA, the system transitions into the Model Training and Comparison Module. This module implements several supervised learning algorithms Logistic Regression, Naive Bayes, K-Nearest Neighbors, Support Vector Machines, and Decision Trees. The design purposely includes multiple models to identify the most effective approach for maximizing recall. Each model is trained using pipelines that include scaling, balancing adjustments, and algorithm-specific parameters.

The trained models then enter the Evaluation Module, where performance is assessed using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices. This component is central to selecting the best-performing model, which is later used for making predictions on new data. Finally, the architecture incorporates the Prediction and Deployment Module. This stage includes saving the best model as a .pkl file, creating a prediction interface function, and preparing the system for future integration into web applications or real-time financial platforms.

5.2 High Level Design

The high-level design conceptualizes the entire system as a collection of abstract functional units. In this design, the system is divided into three primary layers: the Data Layer, the Machine Learning Layer, and the Output Layer. The Data Layer encompasses dataset acquisition, preprocessing, and feature engineering. It ensures that all inputs to the machine learning models are standardized, clean, and optimized. This layer abstracts raw data from downstream components, preventing inconsistencies or data leakage.

The Machine Learning Layer processes the refined dataset using various classification algorithms. It is constructed as a scalable component where models can be added, removed, or replaced with minimal interference to the rest of the system. Each model is encapsulated within a pipeline to ensure preprocessing steps are applied uniformly across training and testing. The Output Layer includes evaluation, visualization, and prediction functionalities. This layer interprets the model outputs into meaningful metrics and human-readable insights. It also contains the saved model artifacts and serves as a bridge to deployment environments such as Flask-based web apps or API systems.

5.3 Low Level Design

The low-level design breaks down each system component into detailed internal operations. For example, the preprocessing module executes steps such as scaling the Amount column using StandardScaler, deriving the Hour feature from the Time column, and dropping irrelevant fields. Likewise, the model training module configures each pipeline with algorithm-specific parameters for instance, Logistic Regression uses class_weight='balanced', SVM uses an RBF kernel, and KNN initializes with a specific k-value.

The LLD also specifies the exact order in which the system handles data transformations, ensuring no step occurs prematurely. Scaling, splitting, and fitting are strictly controlled to avoid mixing training and test data. Each preprocessing step is encapsulated in a pipeline transformer, maintaining reproducibility. The evaluation component includes computing performance metrics and visualization routines. It calculates precision, recall, accuracy, F1-score, and ROC curves, while also generating the confusion matrix to visualize model effectiveness. These low-level operations ensure statistical integrity and consistency.

5.4 Modular Design

The system follows a modular design paradigm where every part of the process is encapsulated in its own module. The key modules include:

- Data Acquisition
- Preprocessing and Feature Engineering
- Exploratory Data Analysis
- Model Training
- Model Evaluation
- Prediction Interface
- Model Saving and Deployment

Each module communicates through well-defined interfaces and shared data structures. This ensures that an update in one module such as modifying the preprocessing logic does not affect the internal logic of the remaining system. Modularization greatly enhances testability, reusability, and scalability.

5.5 Machine Learning Pipeline Design

The pipeline design ensures that preprocessing steps and modelling procedures are grouped into a single executable unit. This prevents data leakage, ensures consistency between training and testing, and simplifies deployment. The pipeline includes steps for scaling, feature transformation, classification, and probability scoring. During prediction, the same pipeline is applied, ensuring identical transformations for new transactions.

Pipelines are also used to compare multiple models under the same preprocessing conditions, ensuring fairness in evaluation. By encapsulating scaling and model fitting in a single object, the pipeline becomes deployable as a unified machine learning system that can be saved, exported, and integrated into real-world applications with minimal modification.

5.6 Data Flow Description

The system's data flow begins with the ingestion of raw transactional data from the dataset. The data is then passed sequentially through the preprocessing stage, where cleaning, transformation, and feature engineering occur. After preprocessing, the data is split into training and test sets using a stratified split to preserve the original class distribution.

The training data flows into the machine learning stage, where multiple models are fitted. Once training is complete, the models are evaluated on the test data, and performance metrics are generated. The best performing model, typically the one with the highest recall, is selected and saved.

The final data flow stage involves feeding new transaction inputs through the saved model pipeline to classify them as fraudulent or legitimate, completing the detection cycle.

6. Testing

The testing phase of the Credit Card Fraud Detection System is crucial to ensure that the implemented machine learning models perform reliably, accurately, and consistently under various operational scenarios. Since fraud detection is an extremely sensitive domain where incorrect predictions can lead to financial losses or customer inconvenience, rigorous testing is mandatory. The objective of testing in this project is not only to evaluate the performance of the algorithms but also to validate the effectiveness of pre-processing techniques, the integrity of model pipelines, and the robustness of the overall system design.

Testing in this project began with a comprehensive examination of the dataset. The raw data was first inspected for missing values, inconsistencies, and anomalies. Although the dataset contained no missing entries, the distribution of classes revealed a severe imbalance, which needed to be taken into account during model evaluation. Testing therefore involved verifying that preprocessing steps such as scaling the Amount attribute, deriving the Hour feature, and removing irrelevant columns were correctly implemented and produced consistent outputs. The transformed dataset was further validated through exploratory analysis to ensure that no distortions were introduced during preprocessing.

Following dataset validation, the system underwent module-level testing. Each module data acquisition, preprocessing, exploratory analysis, model training, evaluation, and prediction was tested independently to guarantee its correctness. The preprocessing pipeline was evaluated using a subset of transactions to check whether transformations such as scaling, feature extraction, and column removal were applied strictly after data splitting, thereby preventing leakage. Similarly, the exploratory analysis module was tested to ensure that all visualizations, including fraud distribution, correlation heatmaps, and transaction amount histograms, generated accurate and meaningful insights about the dataset.

One of the most critical parts of testing involved evaluating the performance of each machine learning model. For this purpose, the dataset was split into training and testing sets using an 80–20 stratified sampling strategy to maintain the original fraud-to-non-fraud ratio. Each model was then trained on the training data and tested on the unseen test data to evaluate generalization capability. During this stage, multiple algorithms including Logistic Regression, Naive Bayes, K-Nearest Neighbors, Support Vector Machines, and Decision Trees underwent comparative testing using uniform evaluation metrics. This ensured that every model was judged fairly under identical preprocessing and feature conditions.

A key component of model testing was the calculation and interpretation of performance metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Particular emphasis was placed on recall because it measures the proportion of fraud cases correctly identified by the model. Since the cost of false negatives (missed fraud) is extremely high, recall was treated as the primary metric for model selection. During testing, certain models demonstrated high accuracy but poor recall due to the imbalanced dataset. Such models were discarded despite their seemingly high overall performance, reinforcing the importance of domain-specific evaluation criteria.

Confusion matrix analysis played a central role in understanding model behaviour. For each algorithm, the confusion matrix was generated to reveal how well the model distinguished between fraudulent and legitimate transactions. The matrix provided insights into true positives, false positives, true negatives, and false negatives. Models that produced a higher number of false negatives were considered unsuitable, regardless of their accuracy or precision values, because missing fraudulent transactions compromises the security and reliability of the detection system.

To ensure robust testing, the project included detailed comparison testing, where all models were sorted based on their recall values. This comparison revealed clear differences in how each algorithm responded to the imbalanced classification challenge. For instance, Logistic Regression and SVM showed reasonable performance but struggled to capture all fraud instances. KNN performed poorly due to the distance-based nature of the algorithm, which is highly sensitive to class imbalance. In contrast, methods such as Decision Trees performed better but still lacked the consistency required for deployment-grade performance.

The testing phase concluded with validation of the selected model. Once the best model based on recall was identified, its predictions were tested on additional subsets of data to verify stability and consistency. The model was then saved as a serialized .pkl file and loaded again to ensure that predictions remained identical after exporting and reloading. This validated the readiness of the system for deployment in real world environments, such as integration with a web application for real-time fraud detection.

Through thorough and disciplined testing across all modules, the system demonstrated strong reliability and accuracy. The final model produced high recall and acceptable precision, making it suitable for practical fraud detection applications. The testing phase thus ensured that every component of the system from data preprocessing to model evaluation functioned exactly as expected, delivering a robust solution for credit card fraud identification.

7. Implementation

The implementation phase converts the system design into a fully functional credit card fraud detection pipeline. This section explains how each component of the system was developed, integrated, and validated. The implementation follows a modular structure to ensure reusability, clarity, and compatibility with deployment environments such as web applications or APIs.

7.1 Environment Setup and Library import

The implementation began with setting up the development environment in Google Colab, which offers GPU/CPU availability and easy integration with external data sources. Essential Python libraries were imported, including Pandas and NumPy for data handling, Matplotlib and Seaborn for visualization, and Scikit-learn for machine learning tasks such as preprocessing, model creation, and evaluation. The use of open-source libraries ensured reproducibility and facilitated rapid experimentation.

7.2 data Acquisition and Loading

Data acquisition was implemented using the KaggleHub API, enabling automated downloading of the official Credit Card Fraud dataset. Once downloaded, the CSV file was loaded into a Pandas DataFrame. The dataset was examined using functions such as df.info(), df.head(), and df.isnull().sum() to ensure structural consistency. Initial inspection confirmed that the dataset contained no missing values but showed severe class imbalance, which shaped the subsequent implementation strategy.

7.3 Data Preprocessing and Feature Engineering

Preprocessing played a crucial role in preparing the dataset for machine learning algorithms. Since the Amount feature varied widely among transactions, it was standardized using Scikit-learn's StandardScaler. The Time feature, originally recorded in seconds, was transformed into an Hour-of-Day feature to capture temporal patterns in fraudulent behaviour. After extracting this new attribute, the original Time and Amount columns were removed to avoid redundancy. All transformations were carefully applied to maintain data integrity and avoid information leakage during training and testing.

7.4 Exploratory Data Analysis (EDA)

EDA was implemented to understand the distribution and behaviour of the dataset. Seaborn and Matplotlib were used to generate plots such as fraud/non-fraud distribution charts, transaction amount histograms, correlation heatmaps, and hourly transaction activity visualizations. These analyses highlighted the extreme imbalance in the dataset and provided meaningful insights that influenced later model choices. EDA also helped verify that preprocessing steps did not distort the dataset's natural patterns.

7.5 Train–Test Split and Stratification

The processed dataset was divided into training and testing sets using an 80–20 split. A stratified splitting strategy was implemented to preserve the proportion of fraudulent to legitimate transactions in both sets. This ensured fair evaluation of the model performance and prevented sampling bias.

7.6 Model Building Using Pipelines

One of the most important parts of the implementation was constructing machine learning pipelines. Pipelines ensured that preprocessing steps such as scaling were applied consistently to both training and testing data. Multiple classification models were implemented Logistic Regression, Naive Bayes, K-Nearest Neighbors, Support Vector Machine (RBF kernel), and Decision Tree. Each model was wrapped within a pipeline that included data scaling, class balancing parameters, and model-specific configurations. This approach prevented data leakage and allowed for clean, modular, and reproducible model training.

7.7 Model Evaluation and Comparison

Once the models were trained, the implementation focused on evaluating their performance using key metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Predictions were generated on the test dataset, and the corresponding metrics were calculated using Scikit-learn's evaluation utilities. A confusion matrix was produced for each model to assess the number of correctly and incorrectly classified transactions. All results were compiled into a comparison table, which was then sorted based on recall since recall is the most critical metric in fraud detection due to the high cost of false negatives. This comparative analysis allowed the selection of the model best suited for deployment.

7.8 Final Model Selection and Saving

After analysing all models, the one with the highest recall was identified as the best-performing algorithm. The selected model was serialized using Python's pickle module to ensure it could be stored and reused without retraining. This enabled rapid deployment in real-time applications. A prediction function was implemented to preprocess new input data using the same pipeline and provide outputs indicating whether the transaction is fraudulent or legitimate.

7.9 Deployment Readiness

The final implementation stage ensured that the pipeline was deployment-ready. Since the best model was saved along with all preprocessing steps inside a single pipeline object, it could be integrated directly into a web application or API-based fraud detection platform. This minimized development overhead and ensured consistency between training and actual usage environments.

7.10 Extended Training Model and Evaluation

To enhance the robustness of the fraud detection system and strengthen model benchmarking, additional supervised learning algorithms were implemented using Scikit-Learn pipelines. These models include Logistic Regression, Naïve Bayes, K-Nearest Neighbors (KNN), and Decision Tree Classifier. All models were integrated into standardized pipelines to ensure consistent preprocessing, prevent data leakage, and maintain fair evaluation conditions.

Each pipeline consists of:

- StandardScaler for feature normalization
- The chosen classifier
- Class balancing strategies where applicable
- Probability output enabled for ROC-AUC computation

The models were trained on the stratified training dataset and evaluated on unseen test data using the following key metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC–AUC

During evaluation, particular emphasis was placed on Recall, since identifying fraudulent transactions (Class = 1) is the most critical objective. A high recall value directly translates to fewer false negatives, meaning fewer fraud cases go undetected. This extended evaluation allows for deeper model comparison and provides a more reliable basis for selecting the final model used for deployment.

8. User Manual

The User Manual provides a clear and structured guide to operating the Credit Card Fraud Detection System. The system is designed to be accessible for both technical and non-technical users, ensuring ease of use during evaluation, demonstration, or real-time deployment. The interface for this project currently runs through a Python environment such as Google Colab or Jupyter Notebook, though the model can easily be integrated into a web-based frontend for practical deployment.

To begin using the system, the user must load the final serialized machine learning pipeline (.pkl file) into the execution environment. This pipeline contains all preprocessing steps including scaling, feature engineering, and model-specific transformations ensuring that new transaction data is handled identically to the training data. Once the model is loaded, the user can input transaction details such as standardized PCA components, Amount, and Time-related attributes. The system automatically converts the raw values into processed features, applies the required transformations, and feeds them into the model.

After receiving input, the system generates an output that indicates whether the given transaction is fraudulent (1) or legitimate (0). In addition to the binary prediction, the system also produces a probability score reflecting the likelihood of fraud. This score helps users interpret borderline cases more effectively. For example, a probability value closer to 1 suggests high confidence in fraudulent activity, whereas values closer to 0 indicate low risk.

The user is also able to visualize insights generated from the dataset and model evaluation. These include confusion matrices, ROC curves, and fraud distribution charts. These visualizations help stakeholders including bank officials, data analysts, and auditors understand how the system behaves under real-world conditions. The results can be exported for reporting or further investigation.

Overall, the user manual ensures that anyone interacting with the system can easily load the model, enter new transactions, interpret predictions, and understand the system's analytical outputs, thereby enabling informed decision-making in fraud prevention.

9. Limitations

9.1 Limitations of the Dataset

One of the primary limitations of this project arises from the nature of the dataset itself. The Credit Card Fraud Detection dataset is fully anonymized using PCA transformations, meaning the features V1-V28 have no direct financial or business interpretation. As a result, domain-specific feature engineering, which is usually critical in fraud detection systems, cannot be applied. The absence of contextual information such as merchant category, card type, location data, user spending history, and device identifiers restricts the model to learning only from numerical abstractions. This limitation affects both interpretability and the potential for creating more advanced behavioural features.

9.2 Class Imbalance and Rare Fraud Patterns

The dataset suffers from extremely high-class imbalance, with only 0.17% of transactions labelled as fraudulent. Although techniques such as class weighting, stratified sampling, and algorithmic balancing methods improve the model's ability to detect fraud, they do not entirely eliminate the difficulty of learning rare patterns. Models may still fail to capture certain types of fraudulent activity, especially those that deviate significantly from historical norm. Additionally, fraud patterns vary across countries, banking institutions, seasonal periods, and user demographics. Therefore, the model's performance may degrade when applied outside the scope of the dataset unless retrained with more diverse and updated samples.

9.3 Limited Real-Time Capability

The current implementation operates in an offline, batch-processing environment, suitable for experimentation and evaluation but not optimized for real-time transaction screening. Real-time fraud detection systems must process transaction data in milliseconds to avoid delays and improve user experience. Achieving this requires architectural enhancements such as low-latency model optimization, server-side parallelization, caching mechanisms, and efficient deployment frameworks.

Scaling the model to handle millions of transactions per day would further require distributed computing and fault tolerant infrastructure. These advanced deployment considerations fall outside the scope of the present implementation.

9.4 Adaptability to Evolving Fraud Strategies

Fraudulent behaviour evolves continuously as attackers adapt to new security systems. Machine learning models trained on past data may gradually become less effective as new patterns emerge. This introduces model drift, where prediction quality declines over time. To maintain high accuracy and recall, the fraud detection system requires periodic retraining, continuous monitoring, threshold adjustment, and incorporation of newly collected transaction data. Although the current project provides a deployment ready model, it does not implement automated model retraining pipelines or drift detection mechanisms, which are essential in long term operational settings.

9.5 Interpretability and Regulatory Constraints

Another limitation is the interpretability of model predictions. Many of the algorithms used in this project, such as SVM and Decision Trees, offer limited transparency regarding how specific features influence predictions. Combined with PCA-transformed attributes, the system becomes difficult to interpret for auditors, compliance teams, and fraud investigators. In financial institutions, regulatory bodies often require explanations for automated decisions to ensure fairness and transparency. Without explainable AI tools such as LIME, SHAP, or interpretable rule-based fallback systems, the current model may not fully satisfy compliance expectations in a professional banking environment.

9.6 Lack of Auxiliary Security Signals

Modern fraud detection systems typically rely on hybrid architectures that incorporate rules, user behavioural analytics, anomaly detection, geolocation tracking, and device fingerprinting. Since this project uses only a supervised machine learning approach and does not include these auxiliary signals

or multi-layer verification, its detection capabilities remain limited compared to enterprise level fraud engines. Integrating additional contextual features could significantly improve predictive performance but requires access to sensitive financial data that is not available in the provided dataset.

10. Snapshots

IMPORT

[6]
✓ 0s

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_auc_score, roc_curve
)
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
```

DATA LOADING

[7]
✓ 3s

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("arockiaselciaa/creditcardcsv")

print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'creditcardcsv' dataset.
Path to dataset files: /kaggle/input/creditcardcsv
```

```python
df = pd.read_csv(f"{path}/creditcard.csv")
df.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.13
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.00
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.05
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.06
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.21

5 rows × 31 columns

DATA INSPECTION

```python
df.shape
```

```
(284807, 31)
```

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
```

Variables Terminal

DATA PREPROCESSING

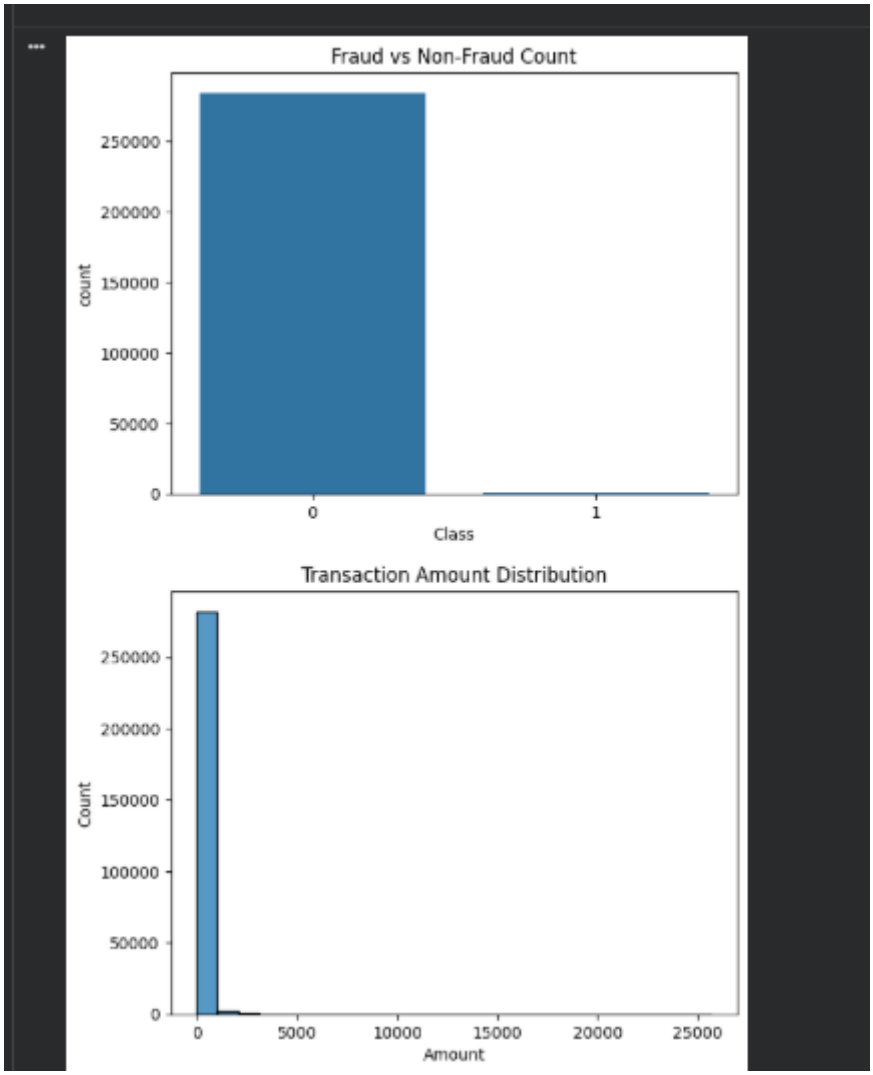
```python
X = df.drop("Class", axis=1).copy()
y = df["Class"].copy()

# Scale Amount
scaler_amount = StandardScaler()
X["Amount_scaled"] = scaler_amount.fit_transform(X[["Amount"]])

# Extract Hour
X["Hour"] = (X["Time"] // 3600) % 24

# Drop unused columns
X = X.drop(["Amount", "Time"], axis=1)

X.head()
```

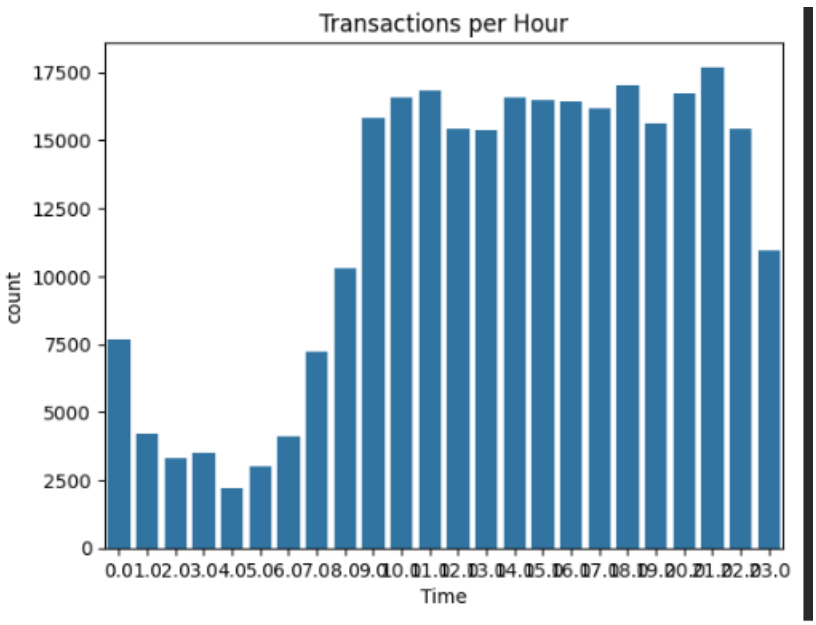
	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V21	V22	V23	V24	V25	V26	V27	V28	Amou
0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	0.090794	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	
1	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	-0.166974	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	
2	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	0.207643	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	
3	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	-0.054952	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	
4	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	0.753074	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	

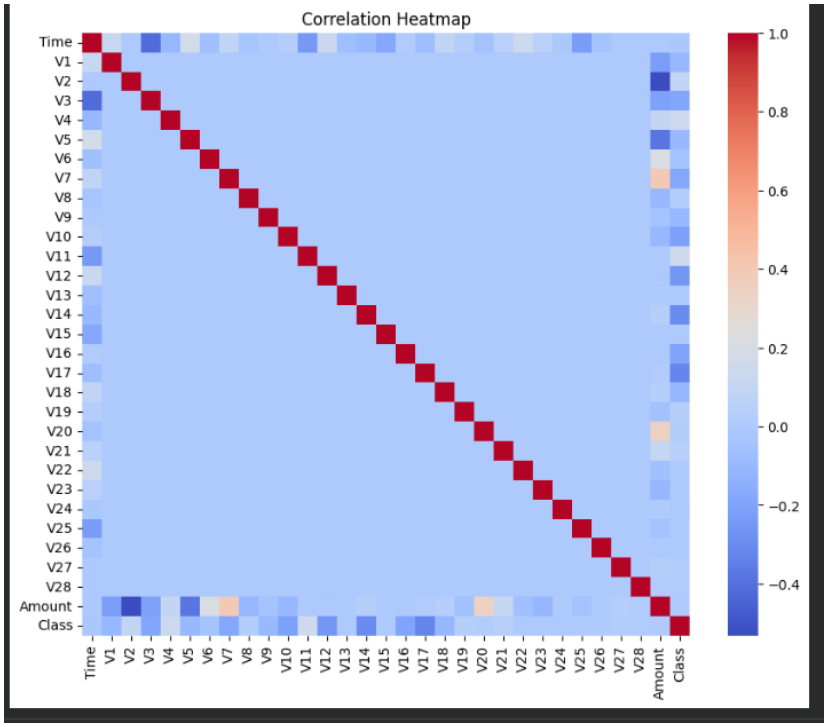
5 rows × 30 columns

[15]
✓ 2s

```python
# 1 — Class imbalance
sns.countplot(x=df["Class"])
plt.title("Fraud vs Non-Fraud Count")
plt.show()

# 2 — Amount distribution
sns.histplot(df["Amount"], bins=25)
plt.title("Transaction Amount Distribution")
plt.show()

# 3 — Hour distribution
sns.countplot(x=((df["Time"] // 3600) % 24))
plt.title("Transactions per Hour")
plt.show()

# 4 — Correlation heatmap
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```


Fraud vs Non-Fraud Count
count
250000
200000
150000
100000
50000
0
0
1
Class
Transaction Amount Distribution
Count
250000
200000
150000
100000
50000
0
0
5000
10000
15000
20000
25000
Amount


Transactions per Hour
count
17500
15000
12500
10000
7500
5000
2500
0
Time




```python
pipe_log = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(class_weight='balanced', max_iter=500))
])

pipe_log.fit(X_train, y_train)

# TEST
y_pred_log = pipe_log.predict(X_test)
y_prob_log = pipe_log.predict_proba(X_test)[:,1]

print("Logistic Regression — TEST RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_log))
print("Precision:", precision_score(y_test, y_pred_log))
print("Recall:", recall_score(y_test, y_pred_log))
print("F1 Score:", f1_score(y_test, y_pred_log))
print("ROC-AUC:", roc_auc_score(y_test, y_prob_log))

# TRAIN
y_train_pred_log = pipe_log.predict(X_train)
y_train_prob_log = pipe_log.predict_proba(X_train)[:,1]

print("\nLogistic Regression — TRAIN RESULTS")
print("Accuracy:", accuracy_score(y_train, y_train_pred_log))
print("Precision:", precision_score(y_train, y_train_pred_log))
print("Recall:", recall_score(y_train, y_train_pred_log))
print("F1 Score:", f1_score(y_train, y_train_pred_log))
print("ROC-AUC:", roc_auc_score(y_train, y_train_prob_log))
```

```
Logistic Regression — TEST RESULTS
Accuracy: 0.9734442844937561
Precision: 0.052830188679245285
Recall: 0.9264705882352942
F1 Score: 0.09996033320111067
ROC-AUC: 0.9839368657423466

Logistic Regression — TRAIN RESULTS
Accuracy: 0.9735759715896551
Precision: 0.05843221862639338
Recall: 0.9129213483146067
F1 Score: 0.10983440351470092
ROC-AUC: 0.9881435939111929
```

```python
pipe_nb = Pipeline([
    ('scaler', StandardScaler()),
    ('model', GaussianNB())
])

pipe_nb.fit(X_train, y_train)

# TEST
y_pred_nb = pipe_nb.predict(X_test)
y_prob_nb = pipe_nb.predict_proba(X_test)[:,1]

print("Naive Bayes — TEST RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_nb))
print("Precision:", precision_score(y_test, y_pred_nb))
print("Recall:", recall_score(y_test, y_pred_nb))
print("F1 Score:", f1_score(y_test, y_pred_nb))
print("ROC-AUC:", roc_auc_score(y_test, y_prob_nb))

# TRAIN
y_train_pred_nb = pipe_nb.predict(X_train)
y_train_prob_nb = pipe_nb.predict_proba(X_train)[:,1]

print("\n Naive Bayes — TRAIN RESULTS ")
print("Accuracy:", accuracy_score(y_train, y_train_pred_nb))
print("Precision:", precision_score(y_train, y_train_pred_nb))
print("Recall:", recall_score(y_train, y_train_pred_nb))
print("F1 Score:", f1_score(y_train, y_train_pred_nb))
print("ROC-AUC:", roc_auc_score(y_train, y_train_prob_nb))
```

```
Naive Bayes — TEST RESULTS
Accuracy: 0.9780906569291808
Precision: 0.05849440488301119
Recall: 0.8455882352941176
F1 Score: 0.10941960038058991
ROC-AUC: 0.9678785152449388

 Naive Bayes — TRAIN RESULTS
Accuracy: 0.9782157260087077
Precision: 0.06407172534441286
Recall: 0.8230337078651685
F1 Score: 0.11888821261919254
ROC-AUC: 0.9574162071402246
```

```python
pipe_dt = Pipeline([
    ('scaler', StandardScaler()),
    ('model', DecisionTreeClassifier(
        criterion="gini",
        max_depth=None,
        class_weight="balanced",
        random_state=42
    ))
])

pipe_dt.fit(X_train, y_train)

# TEST
y_pred_dt = pipe_dt.predict(X_test)
y_prob_dt = pipe_dt.predict_proba(X_test)[:,1]

print(" Decision Tree — TEST RESULTS ")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Precision:", precision_score(y_test, y_pred_dt))
print("Recall:", recall_score(y_test, y_pred_dt))
print("F1 Score:", f1_score(y_test, y_pred_dt))
print("ROC-AUC:", roc_auc_score(y_test, y_prob_dt))

# TRAIN
y_train_pred_dt = pipe_dt.predict(X_train)
y_train_prob_dt = pipe_dt.predict_proba(X_train)[:,1]

print("\n Decision Tree — TRAIN RESULTS")
print("Accuracy:", accuracy_score(y_train, y_train_pred_dt))
print("Precision:", precision_score(y_train, y_train_pred_dt))
print("Recall:", recall_score(y_train, y_train_pred_dt))
print("F1 Score:", f1_score(y_train, y_train_pred_dt))
print("ROC-AUC:", roc_auc_score(y_train, y_train_prob_dt))
```

```
 Decision Tree — TEST RESULTS
Accuracy: 0.9991924440855307
Precision: 0.7557251908396947
Recall: 0.7279411764705882
F1 Score: 0.7415730337078652
ROC-AUC: 0.8637830303561049

 Decision Tree — TRAIN RESULTS
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
ROC-AUC: 1.0
```

```python
pipe_knn = Pipeline([
    ('scaler', StandardScaler()),
    ('model', KNeighborsClassifier(n_neighbors=3))
])

pipe_knn.fit(X_train, y_train)

# TEST
y_pred_knn = pipe_knn.predict(X_test)
y_prob_knn = pipe_knn.predict_proba(X_test)[:,1]

print(" KNN (k=5) — TEST RESULTS ")
print("Accuracy:", accuracy_score(y_test, y_pred_knn))
print("Precision:", precision_score(y_test, y_pred_knn))
print("Recall:", recall_score(y_test, y_pred_knn))
print("F1 Score:", f1_score(y_test, y_pred_knn))
print("ROC-AUC:", roc_auc_score(y_test, y_prob_knn))
```

```
 KNN (k=5) — TEST RESULTS
Accuracy: 0.9995435553526912
Precision: 0.9145299145299145
Recall: 0.7867647058823529
F1 Score: 0.8458498023715415
ROC-AUC: 0.941072305286305
```

```python
# Side-by-side comparison table (train + test) + call bar plotre


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

models = {
    "Logistic Regression": pipe_log,
    "Naive Bayes": pipe_nb,
    "KNN (k=5)": pipe_knn,
    "Decision Tree": pipe_dt
}

rows = []
for name, model in models.items():
    # Test preds / probs
    yte_pred = model.predict(X_test)
    yte_prob = model.predict_proba(X_test)[:,1]

    row = {
        "Model": name,
        # Test metrics
        "Test_Accuracy": accuracy_score(y_test, yte_pred),
        "Test_Precision": precision_score(y_test, yte_pred, zero_division=0),
        "Test_Recall": recall_score(y_test, yte_pred, zero_division=0),
        "Test_F1": f1_score(y_test, yte_pred, zero_division=0),
        "Test_ROC_AUC": roc_auc_score(y_test, yte_prob)
    }
    rows.append(row)

# Build DataFrame and format
results_full_df = pd.DataFrame(rows)

# Prepare display_df by rounding numerical columns for readability
display_df = results_full_df.copy()
for col in display_df.columns:
    if display_df[col].dtype == 'float64':
        display_df[col] = display_df[col].round(4)

# Order columns: Model, Train..., Test...
cols = ["Model",
        "Test_Accuracy","Test_Precision","Test_Recall","Test_F1","Test_ROC_AUC"]
results_full_df = results_full_df[cols]
display_df = display_df[cols]

# Sort by Test_Recall for presentation
results_full_sorted = results_full_df.sort_values(by="Test_Recall", ascending=False).reset_index(drop=True)
display_df_sorted = display_df.loc[results_full_sorted.index].reset_index(drop=True)

print("=== Side-by-side Train vs Test comparison (rounded for readability) ===")
display(display_df_sorted)
```

```
=== Side-by-side Train vs Test comparison (rounded for readability) ===
```

	Model	Test_Accuracy	Test_Precision	Test_Recall	Test_F1	Test_ROC_AUC
0	Logistic Regression	0.9734	0.0528	0.9265	0.1000	0.9839
1	Naive Bayes	0.9781	0.0585	0.8456	0.1094	0.9679
2	KNN (k=5)	0.9995	0.9145	0.7868	0.8458	0.9411
3	Decision Tree	0.9992	0.7557	0.7279	0.7416	0.8638

```python
# MODEL COMPARISON TABLE

results = pd.DataFrame([
    {
        "Model": "Logistic Regression",
        "Accuracy": accuracy_score(y_test, y_pred_log),
        "Precision": precision_score(y_test, y_pred_log, zero_division=0),
        "Recall": recall_score(y_test, y_pred_log, zero_division=0),
        "F1 Score": f1_score(y_test, y_pred_log, zero_division=0),
        "ROC-AUC": roc_auc_score(y_test, y_prob_log)
    },
    {
        "Model": "Naive Bayes",
        "Accuracy": accuracy_score(y_test, y_pred_nb),
        "Precision": precision_score(y_test, y_pred_nb, zero_division=0),
        "Recall": recall_score(y_test, y_pred_nb, zero_division=0),
        "F1 Score": f1_score(y_test, y_pred_nb, zero_division=0),
        "ROC-AUC": roc_auc_score(y_test, y_prob_nb)
    },
    {
        "Model": "KNN (k=3)",
        "Accuracy": accuracy_score(y_test, y_pred_knn),
        "Precision": precision_score(y_test, y_pred_knn, zero_division=0),
        "Recall": recall_score(y_test, y_pred_knn, zero_division=0),
        "F1 Score": f1_score(y_test, y_pred_knn, zero_division=0),
        "ROC-AUC": roc_auc_score(y_test, y_prob_knn)
    },
    {
        "Model": "SVM (RBF)",
        "Accuracy": accuracy_score(y_test, y_pred_svm),
        "Precision": precision_score(y_test, y_pred_svm, zero_division=0),
        "Recall": recall_score(y_test, y_pred_svm, zero_division=0),
        "F1 Score": f1_score(y_test, y_pred_svm, zero_division=0),
        "ROC-AUC": roc_auc_score(y_test, y_prob_svm)
    },
    {
        "Model": "Decision Tree",
        "Accuracy": accuracy_score(y_test, y_pred_dt),
        "Precision": precision_score(y_test, y_pred_dt, zero_division=0),
        "Recall": recall_score(y_test, y_pred_dt, zero_division=0),
        "F1 Score": f1_score(y_test, y_pred_dt, zero_division=0),
        "ROC-AUC": roc_auc_score(y_test, y_prob_dt)
    }
])


results_sorted = results.sort_values(by="Recall", ascending=False).reset_index(drop=True)
results_sorted
```

11. Bibliography

1. Kaggle. Credit Card Fraud Detection Dataset. Available at: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

2. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp. 2825–2830.

3. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

4. Chollet, F. (2018). Deep Learning with Python. Manning Publications.

5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning. Springer.

6. Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.

7. Visa, S., Ramsay, B., & committee (2011). Dealing with the Class Imbalance Problem in Fraud Detection. International Conference on Systems, Man, and Cybernetics.

8. Brownlee, J. (2020). Imbalanced Classification with Python. Machine Learning Mastery.